LABORATORIO DE Programación Orientada a Objetos

UNIDAD ACADÉMICA:	Ingeniería de Sistemas
CURSO:	Estructuras de Datos
PRACTICA Nº XX:	1

1. OBJETIVOS

- Aplicar conceptos fundamentales de estructuras de datos mediante la implementación de un monedero de criptomonedas en C++, utilizando vectores y otras estructuras de datos para almacenar y gestionar monedas y transacciones de manera eficiente.

- Desarrollar la habilidad de diseñar e implementar arquitecturas de clases que hagan uso de estructuras de datos para resolver problemas complejos, como el manejo y la manipulación de transacciones y saldos dentro de una aplicación de criptomonedas.

- Simular y analizar la eficiencia de diferentes operaciones relacionadas con la gestión de datos en un monedero de criptomonedas, tales como la inserción, búsqueda y actualización de transacciones, aplicando técnicas de optimización en el uso de estructuras de datos.

2. CONSULTA PREVIA

Los temas esenciales para complementar esta práctica son:

- Programación orientada a objetos: herencia simple y múltiple, clases abstractas e interfaces y polimorfismo, sobrecarga de operadores.
- Estructuras de datos de arreglos usando vectores.
- Diagrama de clases UML.

3. FUNDAMENTO TEÓRICO

La programación orientada a objetos es un paradigma de programación (una técnica o forma de programar), una forma que nosotros podemos resolver un problema usando Clases y Objetos. Esta compuesto por cuatro elementos y se fundamenta en cuatro pilares.

Elementos:

- Clases.
- Propiedades.
- Métodos.
- Objetos.

Pilares:

- Encapsulamiento
- Abstracción.
- Herencias
- Polimorfismo.

Objetos:
Es una entidad que tiene características que lo hacen diferente a otros, también conocidos como propiedades y puede tener acciones diferentes (métodos).

LABORATORIO DE Programación Orientada a Objetos

Clases:
Una clase es el modelo por el cual nuestros objetos se van a construir y nos van a permitir generar más objetos.

Encapsulamiento:
Su significado es asegurarse de que los datos sensibles estén ocultos a los usuarios. La encapsulación significa el proceso de empaquetar los datos y las funciones en una sola cápsula. También protege los datos de otras clases limitando el acceso. Básicamente, oculta los datos.

Herencia:
Definiéndose de una manera sencilla como la transmisión de código fuente entre unas clases y otras. La herencia nos permite que una clase pueda heredar sus atributos y métodos. Tenemos varios tipos de herencia: simple, multinivel, jerárquica y múltiple.

Clases abstractas:
A veces, la implementación de algunas funciones no se puede proporcionar en una clase Padre porque no conocemos su implementación. Esta clase se llama clase Abstracta.

Clases interfaces:
Una interfaz no tiene implementaciones de ninguno de sus métodos, se puede considerar como una colección de declaraciones de métodos.

Polimorfismo:
A menudo se conoce como el tercer pilar de la programación orientada a objetos, después de la encapsulación y la herencia. Polimorfismo es una palabra griega que significa “muchas formas”. El polimorfismo lo vemos reflejado en muchas cosas. Por ejemplo decimos el verbo abrir, que significa abrir la puerta, una caja, un libro.

4. EQUIPOS, MATERIALES Y REACTIVOS

Equipos	Materiales	Sustancias y/o Reactivos
Computadores	Software: IVisual Studio code	

5. PROCEDIMIENTO O METODOLOGÍA

Los pasos a seguir en la práctica son los siguientes:

- Integrantes: Trabajo en grupo (2 personas).
- Lugar: Sala de laboratorio.
- Tiempo de trabajo: 1 hora y 30 minutos de clase en el laboratorio.
- Actividad: Leer la guía del laboratorio, desarrollar la actividad propuesta durante la sesión en el laboratorio, presentar y sustentar la solución al docente. Si es necesario, subir la solución al medio designado para cada laboratorio.
-

6. RESULTADOS

Realizar un programa en C++ que consiste en un **Monedero de Criptomonedas,** este monedero tendrá la posibilidad de crear monedas y realizar transacciones de dos tipos: para enviar y recibir dineros. La arquitectura

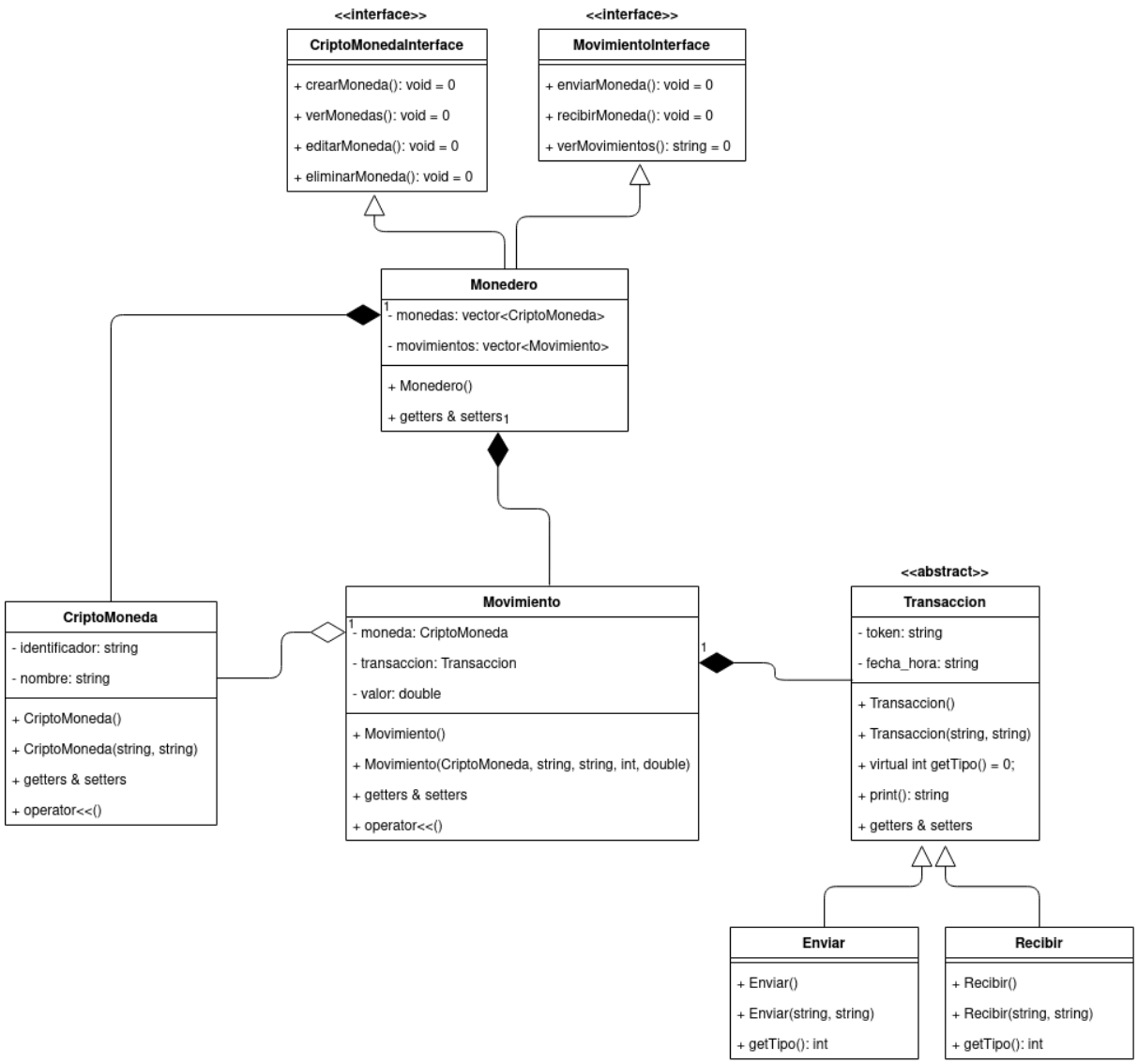
LABORATORIO DE Programación Orientada a Objetos

de las clases se encuentra en el diagrama UML adjunto. Tener en cuenta lo siguiente:

1. Crear una clase CriptoMoneda con dos atributos: nombre completo de la moneda y el identificador para la abreviación internacional, por ejemplo bitcoin es BTC. Esta clase debe cumplir con el encapsulamiento y la sobrecarga del operador ostream <<.
2. Hay dos tipos de transacciones: para enviar moneda y recibir moneda. La transacción es una clase abstracta que contiene los atributos: fecha y hora de la transacción y el token es el identificador único e irrastreable, generalmente un código alfanumérico como por ejemplo eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9. Esta clase tiene dos constructores, un método print y un método virtual puro. Posteriormente crear las clases Enviar y Recibir, y en ellas implementar el método getTipo del padre, este método debe retornar un entero, 1 para enviar y 2 para recibir. No olvidar hacer encapsulamiento.
3. Crear la clase Movimiento que tiene la moneda, la transacción realizada y el valor de la transacción. Tener en cuenta que los atributos moneda y transacción son asociaciones, para el constructor con parámetros recuerde que existen dos tipos de asociaciones, por eso se agrega una variable de tipo entero para validar si se requiere crear un movimiento “Enviar (1)” o “Recibir (2)”. No olvidar realizar encapsulamiento y sobrecarga del operador ostream <<.
4. Crear una Clase Monedero que será donde estará toda la lógica del programa con el registro de monedas y el registro de movimientos realizados. Esta clase implementa (hereda) los métodos de las clases interfaces CriptoMonedaInterface y MovimientoInterface. También se requiere realizar encapsulamiento.
5. Para el funcionamiento del monedero crear 3 monedas y agregarlas al atributo vector<CriptoMoneda>: bitcoin (BTC), etherium (ETH) y ripple (RIPPLE). Realizar 10 transacciones: 4 de Enviar monedas y 6 de Recibir monedas (utilice todas las monedas). Todas las transacciones se deben guardar en el vector<movimiento> de la clase Monedero. Al final deben calcular el saldo de cada moneda según los movimientos realizados (vector<movimiento>), ejemplo: Si recibí 10 monedas de BTC y envié 4 monedas, el saldo final de mis monedas BTC en el monedero es de 6 monedas BTC.
6. Se evalúa el código limpio y funcionalidad sin errores. Se requiere realizar separación de archivos en .h y .cpp por cada clase.

LABORATORIO DE Programación Orientada a Objetos

Diagrama UML



7. BIBLIOGRAFIA

C++ Programming: An Object-Oriented Approach. McGraw-Hill Education (2019). Behrouz A. Forouzan, Richard Gilberg.